

# Ctrl-Alt-Eat

Golden Prompts Documentation

Kitchen Display & Customer Ordering System

*CSE323: Software Engineering*  
*Egypt-Japan University of Science and Technology (E-JUST)*

| ID        | Name             | Contribution                       |
|-----------|------------------|------------------------------------|
| 120230008 | Adnan Ahmed      | Full-Stack Scaffolding & UI/UX     |
| 120230073 | Abdelrahman Adel | Monorepo & API Architecture        |
| 120230062 | Ahmed El-Shazly  | Order Tracking & Checkout          |
| 120230056 | Mohamed Ashraf   | Chef KDS System                    |
| 120230091 | Seif Fayed       | Frontend Auth & Ordering Lifecycle |

## Table of Contents

---

1. System Context & Overview
2. Adnan Ahmed (120230008) — Full-Stack Scaffolding & UI/UX
  - Prompt 1: Full-Stack Project Scaffolding & Customer Browsing
  - Prompt 2: High-Fidelity UI/UX Revamp & Brand Theme
  - Prompt 3: Animations & Dark/Light Mode
3. Abdelrahman Adel (120230073) — Monorepo & API Architecture
4. Ahmed El-Shazly (120230062) — Order Tracking & Checkout
5. Mohamed Ashraf (120230056) — Chef KDS System
  - Design Blueprint
  - Implementation
6. Seif Fayed (120230091) — Frontend Auth & Ordering Lifecycle

---

# 1. System Context & Overview

---

## System Overview

---

Ctrl-Alt-Eat is a modern, real-time, full-stack web application designed to streamline restaurant operations. It unites front-of-house customer ordering with back-of-house kitchen display workflows. The platform is designed around two core personas: Customers who want to customize and track orders in real time, and Chefs who need a high-performance, clutter-free queue to manage preparation workflows.

The system is built using a Django REST Framework (DRF) backend, a React.js + Tailwind CSS frontend, and Django Channels (WebSockets) to handle real-time, bi-directional state transitions.

## Architectural Stack

---

- Frontend SPA: React.js — Single Page App, optimized state handling via Context APIs, fast visual responses using CSS & Tailwind.
- API Server: Django REST Framework — Modularized viewsets for CRUD operations, JWT/Token authorization for route validation, built-in model constraints.
- Real-time Pipeline: Django Channels — ASGI server wrapper running Daphne, managing in-memory message channels for order-ready broadcasts.
- Database: SQLite — Serverless development database with custom validators on integer boundaries (table numbers 1-100).
- Modularity & Quality: Playwright & Pytest — Test suite covering 18+ backend tests and End-to-End Master Journey flows.

## Core Functional Workflows

---

### Authentication & Intelligent Role Detection

The entry point is a Login & Sign-Up Page. During user registration, the system inspects the email domain. If the email matches the institutional suffix @ejust.edu.eg, the user is automatically designated with the Chef role. Any other email domain defaults to the Customer role.

### Customer Ordering Journey

The Customer Portal focuses on a friction-free customization and checkout experience. Features include structured menu browsing across three categories (Main Meals, Desserts, Drinks), granular add-on customizations, persistent multi-category shopping cart, simulated checkout gateway, and real-time preparation tracking via WebSocket toast notifications.

## Chef Kitchen Display System (KDS)

The Chef Portal is a real-time command board with FIFO queue organization, detailed order cards (order number, table, elapsed time, items, add-ons, notes), and single-action workflow buttons (Start Preparation, Mark as Ready, Mark as Delivered).

---

## 2. Adnan Ahmed (120230008)

---

### Full-Stack Scaffolding, UI/UX Revamp & Interactive Animations

Adnan's prompts cover the complete initialization of the system from scratch, including backend/frontend structures, role detection, menu browsing, persistent shopping cart, mock checkout, and three successive rounds of UI polish — brand theming, micro-animations, and cohesive dark/light mode.

## Prompt 1: Full-Stack Project Scaffolding & Customer Browsing Slice

---

Role: Senior Full-Stack Architect & Principal Software Engineer

Task Overview:

Initialize a full-stack, modular Kitchen Display System (KDS) using Django (DRF) and React (Tailwind CSS). Build the first major vertical slice: Authentication, Account Type Resolution, Grouped Category Menu Browsing, and a Persistent Multi-Category Shopping Cart.

### Strict Technical Constraints

- Directory Structure Integrity: Adhere strictly to the backend/frontend split and folder layout.
- Decoupled Architecture: Ensure the backend enforces clear RESTful API contracts.
- Explicit Database Constraints: Use Django model validators to ensure numeric values fall within logical bounds.
- Clean Code Quality: Write descriptive variables, include brief docstrings, enforce strong linting.

### 1. Backend Scaffolding & Intelligent Authentication

- Set up the Django project configuration and orders application.
  - Implement standard email and password authentication using DRF Token Authentication.
  - Create a signup serialization pipeline that parses the registering user's email domain.
- If the email ends with @ejust.edu.eg, automatically assign the role of 'chef'.
- For all other email domains, default the user role to 'customer'.
- Restrict orders-related views using customized permissions.

### 2. Frontend Layout & Dynamic Role Routing

- Scaffold the React application using Vite/CRA.
- Implement a global AuthContext to store session token, user details, and active roles, persisting state to localStorage.
- Construct a Login/Sign-Up portal. Once authenticated, dynamically redirect Customers to /menu, Chefs to /chef.
- Use React Router route guards (ProtectedRoute) to strictly block unauthorized access.

### 3. Customer Menu Browsing (Category Selector)

- Create a grouped category menu display divided into three distinct tabs: Main Meals, Desserts, and Drinks.
- Fetch menu items dynamically from the GET /api/menu-items/ endpoint.
- For each item card, render the item title, description, price, and custom add-on chips/checkboxes.

### 4. Global State Shopping Cart (Cross-Category & Persistent)

- Design a CartContext provider that persists items across routing transitions.
- Provide a persistent Cart UI showing selected items, custom add-ons, quantities, subtotals, and grand total.
- Table number input with active validations (table 1 to 100).
- Wire the "Place Order" button to POST /api/orders/.

### 5. Chef Dashboard Wireframe & Mocks

- Design a read-only dashboard UI for chefs, fetching active orders using GET /api/dashboard/ in FIFO order.
- Each order card should display table number, timestamp, full customized item list, and a "Mark as Ready" action button.
- Implement a mock WebSocket alert hook for order-ready notifications.

## Prompt 2: High-Fidelity UI/UX Revamp & Brand Theme Alignment

---

Role: Lead UI/UX Architect & Principal Frontend Engineer

Task Overview:

Refactor the entire frontend layout, aesthetic, and color scheme. Align the application's visual system with the Ctrl-Alt-Eat brand logo color scheme. Elevate the interface from a basic development draft into a premium, state-of-the-art consumer experience.

### Strict Boundaries

- Frontend Only: Do not modify any Django backend, database models, or API endpoints.
- Contrast & Accessibility: Ensure all text elements are highly readable.

### 1. Logo-Inspired Color Palette & Theme Setup

- Analyze the brand logo and extract key brand colors (primary, accent, background neutral, text neutral).
- Map these colors into the global Tailwind config file or as custom CSS HSL variables.
- Use smooth gradients and premium neutral backdrops.

### 2. Welcoming Login Experience

- Refactor the auth page from a generic development layout into a stunning landing portal.
- Replace technical sentences with elegant, high-impact branding statements.
- Center the official Ctrl-Alt-Eat branding with the brand logo prominently displayed.

### 3. Header Navigation & Brand Consolidation

- Remove any simple emoji icons or raw text representations.
- Embed the official brand logo cleanly in the top navigation bar across all pages.

#### 4. Category Listings & Item Card Enhancements

- Remove boring emojis used as category markers or menu icons.
- Replace them with clean, modern SVGs or Lucide icons.
- Redesign the menu items layout with card designs, drop shadows, hover animations, and elegant spacing.

#### 5. High-Fidelity Payment Mockups

- Replace generic text options or basic emojis for payment channels.
- Implement high-fidelity, colored SVG vector logos for Apple Pay, Google Pay, and Credit Cards.
- Design interactive payment option chips that clearly highlight upon user selection.

## Prompt 3: Isolated Branding, Micro-Animations & Persistent Dark/Light Mode

---

Role: Senior Frontend Specialist & Interactive Animations Engineer

Task Overview:

Polish the interactive layers of the application. Focus on three key quality updates: integration of the background-free isolated brand logo, implementation of fluid micro-animations, and fixing the global Light/Dark mode theme toggle.

#### Strict Boundaries

- Frontend Only: Absolutely no modifications to backend business logic, database migrations, or serialization layers.
- Single-Language Copy: Keep all application copy strictly in English.

#### 1. Background-Free Branding Integration

- Use the provided isolated logo with transparent background.
- Replace all instances of the old logo with this isolated version.
- Enforce strict size-scaling constraints for clear rendering in both Light and Dark modes.

#### 2. Micro-Animations & Transitions (CSS & React)

- Hover scale effects on menu cards and action buttons.
- Page-transition slide-ins when navigating between categories or checking out.
- Soft fade effects on modal prompts, toast notifications, and drop-down menus.
- Loading skeletons or spinning SVG loaders for network fetching states.

#### 3. Global Light & Dark Mode Cohesion

- Fix the dark/light mode toggle system to operate cohesively across the entire application shell.
- Ensure switching modes updates all background blocks, navigation headers, sidebar borders, cards, and dropdown menus.

- Define a central theme provider in App.jsx. Ensure the toggle updates the class="dark" attribute on the html or body element.
- Dark Mode: Deep carbon/obsidian backgrounds with crisp white/silver text and vibrant accent borders.
- Light Mode: Soft warm grey or porcelain backgrounds with deep charcoal text and harmonious brand accent boundaries.



---

### 3. Abdelrahman Adel (120230073)

---

## Monorepo Foundation Setup, Decoupled Core Architectures & API Mappings

Role: Lead Full-Stack Architect & Core Infrastructure Engineer

Task Overview:

Build upon the established, fully integrated foundations of the Kitchen Display System. Modularize the core backend API controllers, establish high-fidelity endpoint-to-frontend wiring, and enforce strict end-to-end security layers, ensuring all view classes are cleanly distributed according to single-responsibility patterns.

#### Strict Architectural & Integration Constraints

- Modular View Distribution: Endpoint controllers must be strictly distributed across designated view files.
- High-Performance Decoupling: Use serialization pipelines to isolate API payloads from direct database column names. Build lightweight views optimized for high-performance index scanning (FIFO).
- Secure Context Authentication: Restrict critical operations by checking user tokens, enforcing role-based permissions (waiter, chef, customer), and blocking illegal transitions.
- Seamless Frontend Bindings: Wire all REST endpoints cleanly using a centralized Axios client wrapper.

#### 1. Backend Layer — Clean Views Separation & API Boundaries

General Menu & Order CRUD views (`order_views.py`):

- Implement full CRUD handlers for orders and catalog menu items.
- Enforce permissions so customers can only access menu views and retrieve their own orders.

KDS Kitchen Feeds (`dashboard_views.py`):

- Design high-performance queries returning active orders ordered strictly by arrival sequence (FIFO).
- Use Django optimizations (`select_related` and `prefetch_related`) to prevent N+1 queries during feed fetch loops.

State Transition Triggers (`status_views.py`):

- Build endpoints dedicated purely to updating order statuses (e.g., from `in_progress` to `ready`).
- Implement permission controls ensuring only authorized kitchen staff (chefs) can call these endpoints.

#### 2. Frontend Layer — State Integration & View Wiring

Centralized API Client (`api.js`):

- Integrate a unified Axios client that injects authorization tokens and maps backend endpoints to frontend request loops.

Real-Time KDS Feed (`Dashboard.jsx` & `KitchenBoard.jsx`):

- Connect the kitchen display board to consume `GET /api/dashboard/` and status trigger endpoints.
- Render beautiful, real-time updated order cards displaying customized add-ons, timestamps, table numbers, and statuses.

Order Placement (OrderForm.jsx):

- Map form inputs (table selection, items, and custom instructions) to the nested serialization schema.
- Fire POST request payloads smoothly upon completion.

### 3. Verification & Security Rules

- Unauthorized users (e.g., customers) receive 403 Forbidden when accessing the Chef dashboard.
- Valid status steps are verified successfully; out-of-order steps are rejected.

---

## 4. Ahmed El-Shazly (120230062)

---

### Order Tracking & Atomic Checkout

Role: Senior Full-Stack Engineer & API Architect

Task Overview:

Design and implement a robust, high-performance Order Tracking and Atomic Checkout feature. Ensure order submissions are database-atomic, database schemas are decoupled from public API contracts, and the customer-facing real-time tracking interface operates efficiently under high-frequency polling without memory leaks.

#### Strict Technical & Architectural Constraints

- Atomicity: Every database write involving an Order and its nested OrderItems/AddOns must be enclosed in a single `transaction.atomic()` context. Partial creations (orphaned orders) are strictly unacceptable.
- Decoupled Contracts: Decouple database naming conventions from public API responses. The internal database field `order_status` must be aliased to `status` in all serializations and API payloads.
- Database Performance: Enforce efficient database indices on fields frequently targeted by polling filters (`status` and `priority`) to guarantee fast, non-blocking index lookups.
- Resilient Lifecycle Management: Implement explicit cleanup routines on the frontend. Polling timers must be cleared immediately upon component unmounting or when the target state ('READY') is achieved.

#### 1. Backend Layer — Atomic Persistence & Decoupled APIs

Database Schema & Indexing (`models.py`):

- Define the Order model with an inner `Status(models.TextChoices)` class supporting PREPARING (default) and READY states.
- Include an `IntegerField` for priority and a `table_number` (default 1).
- Add explicit database indexes (`db_index=True`) on `status` and `priority` to facilitate rapid index-scans during status polling.
- Define `OrderItem` and `AddOn` models related to `Order` via foreign keys.

Nested Serialization Pipeline (`serializers.py`):

- Construct a customized `OrderSerializer` that accepts nested `OrderItem` lists, each carrying nested `AddOn` arrays in a single transaction payload.
- Override the `create()` method to dynamically parse the payload, instantiate the relations, and commit them inside an explicit `transaction.atomic()` context block.
- Alias `order_status` to a public-facing read-only "status" field, maintaining DB-to-API decoupling.

Granular API Views (`views/`):

- Implement `order_views.py` to handle `POST /api/orders/` for secure, validated atomic creation. Return the created order id and aliased status.
- Implement `status_views.py` to handle `GET /api/orders/pk/` for order tracking. Expose only the order ID and the aliased status.

## 2. Frontend Layer — State Persistence & Interactive Tracking

Application State Provider (CartContext.jsx):

- Implement a global CartContext to wrap the application, managing selected food items, add-ons, quantities, and totals.
- Expose clean utility handles: addItem, removeItem, clearCart, and total to eliminate prop-drilling.

Centralized API Connector (api.js):

- Define a unified Axios service instance with dedicated methods: orderAPI.placeOrder(payload) and orderAPI.getStatus(orderId).

Atomic Checkout Screen (CheckoutPage.jsx):

- Map the local cart state and chosen add-ons into the exact nested JSON structure expected by the backend.
- Handle loading and network error states gracefully. On successful submission, clear the local cart state and programmatically redirect the user to /track/:id.

Dynamic Order Tracking Page (OrderTrackingPage.jsx):

- Parse the active id parameter from the URL.
- While in PREPARING state, render a glowing preparation icon utilizing a custom, smooth pulse-glow transition.
- When transitioned to READY, display a celebratory completion icon with an energetic bounce animation.

Lifecycle-Safe Polling Component (OrderStatusPoller.jsx):

- Implement a non-rendering utility component to poll GET /api/orders/:id/ status every 4 seconds.
- Pass the status changes up via an onStatusUpdate callback handler.
- CRITICAL: Ensure the polling setInterval is cleanly disposed (clearInterval) when the component unmounts OR when the retrieved status transitions to READY.

Fullscreen Ready Notification Modal (ReadyNotification.jsx):

- Create a high-z-index (z-50), fixed, full-screen notification overlay.
- Style it with elegant backdrop blurs (backdrop-blur-md), a rich dark semi-transparent overlay, and modern typography.
- Render this overlay exclusively when the tracking status is updated to READY.

## 3. Styling & Custom Animations (index.css)

- Set up custom theme variables for primary, background, and accent colors.
- Define @keyframes rules for pulse-glow (soft, rhythmic opacity and shadow expansions) and bounce (clean, energetic vertical translations with custom cubic-bezier timing).
- Expose modern utility classes such as .gradient-text and .animate-pulse-glow.

---

## 5. Mohamed Ashraf (120230056)

---

### Chef / Kitchen Display System (KDS)

---

## Prompt 1: KDS Architectural Design Blueprint

---

Role: Senior Software Architect

Task Overview:

Design the complete full-stack Chef / Kitchen Display System (KDS) feature from scratch. The scope is limited strictly to backend and frontend KDS operations, starting immediately after an order is successfully submitted.

### Strict System Boundaries

- Chef-Specific Bounds: The Chef role must not create orders, manage customer ordering, or handle payment checkouts. The KDS operates solely on completed active orders.
- Design Only: Do not modify any codebase files or write runtime source code in this step.

### 1. Recommended Folder/File Structure

- Detail how the KDS slice integrates into the existing directory layout, specifying views, serializations, components, pages, and tests.

### 2. Backend Logic Design

- Define the Django views, permission classes, and querysets needed to drive the KDS APIs.

### 3. Frontend Screen & Component Hierarchy

- Diagram the React layout for the KDS, detailing the parent dashboard views and modular child components.

### 4. API Contract Specification

- GET /api/dashboard/ — Retrieve active orders in FIFO order (payload models, headers, HTTP status codes).
- PATCH /api/orders/{id}/ — Transition order status.

### 5. Data Model & State Assumptions

- List the fields, types, indexes, and relationship attributes required on the Order, OrderItem, and AddOn schemas.

### 6. State Management Approach

- Define the state handling mechanisms on the React frontend (contextual global stores vs local hooks, refresh caching, update overlays).

### 7. FIFO Queue Logic

- Define database sorting rules and timestamp tracking models to guarantee strict First-In, First-Out sequence.

## 8. Order Status Transition Rules

- Formal state-machine defining valid transitions: Incoming -> Preparing -> Ready.
- Explicitly detail rejection responses for out-of-order or duplicate transition attempts.

## 9. Error, Loading & Empty States

- Custom visual layouts and response paradigms for loading delays, blank order queues, and API network dropouts.

## 10. Edge Cases Analysis

- Handling protocols for simultaneous kitchen updates, stale frontend card data, and invalid order index references.

## 11. Comprehensive Testing Plan

- Backend (unit/integration), Frontend (components), Integration (end-to-end), and Performance/Stress (simultaneous updates).

# Prompt 2: KDS Full Implementation

---

Role: Principal Full-Stack Engineer

Task Overview:

Implement the complete, high-fidelity Chef / Kitchen Display System (KDS) vertical slice from scratch. Build the backend views, serialization guards, frontend dashboard pages, interactive status controls, and full-scale testing suites.

## Strict Development Constraints

- Version Control: Verify and confirm that the current workspace branch matches the dedicated development branch before committing edits.
- Isolated Chef Scope: Do not implement ordering features, customer menus, or checkout modules.
- Architecture Style: Adhere strictly to the existing code guidelines and modular design standards.
- State Transitions: Enforce robust state-machine boundaries to reject illegal status steps.

## 1. Backend Layer — FIFO Active API & Transition Guards

Active Kitchen Dashboard Endpoint (GET /api/dashboard/):

- Implement a dedicated view returning active kitchen orders in FIFO order based on submission timestamp.
- Serialize detailed order cards: order ID, table number, ordered items, quantities, customized add-ons, special customer instructions, timestamp, status, and estimated prep time.

Order Status Update Endpoint (PATCH /api/orders/{id}/):

- Implement dynamic status modification logic.
- Enforce strict state transition rules: Incoming -> Preparing -> Ready.

- Validate and reject invalid status transitions with an explicit HTTP 400 Bad Request.

## 2. Frontend Layer — Real-Time Chef Board

Kitchen Dashboard Page (Dashboard.jsx):

- Fetch active orders on load and establish clean polling or WebSocket updates.
- Display cards in a scrollable, high-contrast column using the logo brand theme.
- Each order card must clearly render order ID, table number, elapsed time, item list, quantities, add-ons, and customer instructions.

Workflow Action Buttons (StatusButtons.jsx):

- Integrate a single-action button on each card to transition the order to the next sequential status.
- Provide inline loaders and disable interactions during active API calls to block duplicate trigger clicks.

## 3. Resilient UI States

- Loading: Smooth modern skeletons or spinner icons.
- Empty Queue: A friendly branding message indicating all orders are completed.
- API Error: Accessible banner warnings showing network failure states.
- Update-In-Progress: Half-opacity button cards while saving status.
- Failed Update Alert: A clear popup overlay if an API transition is rejected.

## 4. Edge Cases Mitigation

- Prevent duplicate status submissions by disabling actions immediately after click.
- Prevent stale UI displays by handling conflict resolutions if two kitchen displays access the same order.
- Handle invalid ID routing gracefully with 404 overlays.

## 5. Automated Testing Suites

- FIFO Database Sorting: Verify order lists retrieve the oldest active tickets first.
- Successful Status Step: Validate transitions through Incoming -> Preparing -> Ready.
- Transition Boundaries: Assert validation failures and HTTP 400 responses for incorrect sequences.
- Endpoint Health: Test mock API queries for GET /api/dashboard/ returning empty vs populated datasets.
- Frontend UI Coverage: Verify error state banner representations and button disable triggers.

## 6. Seif Fayed (120230091)

### High-Performance Frontend Authentication & Modular Ordering Lifecycle

Role: Principal Frontend Engineer & System Integrator

Task Overview:

Design and implement a highly polished, modular, and responsive Authentication & Ordering Frontend Flow for the Ctrl-Alt-Eat restaurant application. Enable customer registration, intelligent domain-based role mapping, multi-category browsing with persistent state transitions, simulated checkouts, and active tracking boards.

#### Strict Boundaries & Constraints

- Pure Frontend Scope: Do not alter, edit, or refactor any backend Python code, models, or database schemas.
- Architecture & File Placement: Add pages to `pages/`, shared components to `components/`, and state providers to `context/`.
- Non-Intrusive Integration: Limit changes in root system files (such as `App.jsx`) to routing declarations and provider wrappers.
- Robust State Isolation: Do not allow shopping cart state to leak or reset during UI category switching or back-navigation transitions.

#### 1. Unified Authentication Portal (Login.jsx)

Interactive Auth Form:

- Create a welcoming, high-contrast auth page containing fields for Email and Password.
- Implement a dual-action interface supporting both Login and Sign-Up.

Intelligent Domain-Based Role Detection:

- If the domain matches the official E-JUST suffix `@ejust.edu.eg`, assign the user role of 'chef'.
- For all other domains, assign the user role of 'customer'.

Role-Based Protected Redirections:

- Direct authenticated Chefs to the Kitchen Dashboard (`/chef`).
- Direct authenticated Customers to the menu browsing home page (`/menu`).
- Wrap pages with role-sensitive route guards to prevent unauthorized side-loading.

#### 2. Customer Menu Browsing (CreateOrder.jsx)

Grouped Category Dashboard:

- Provide customers with three explicit categorization cards/tabs: Main Meals, Desserts, and Drinks.
- Render an interactive, scrollable list of food items under each active category.

Granular Customization Option Chips (Add-ons):



- Bind custom add-on selections (e.g., extra sauce/cheese for meals, ice levels for drinks) directly onto each item card.

Persistent Cross-Category State Flow:

- Design the browse loop so that adding an item to the cart, navigating to another category, or pressing the browser/UI "Back" button never wipes or resets the active cart contents.

### 3. Unified Shopping Cart (Cart.jsx)

Detailed Order Manifest:

- Display all selected items, quantities, and customized add-ons.
- Present clear line-item subtotals alongside a responsive grand total.

High-Fidelity Simulated Payments (PaymentOptions.jsx):

- Implement a modern mock payment panel showing selectable, colored options for Visa/Credit Card and Digital Wallets.
- Selecting an option must dynamically complete the order successfully without requiring an active payment server connection.

### 4. Interactive Order Tracking (OrderTrackingPage.jsx)

Live Status Progression UI:

- Provide a clean post-checkout interface displaying: "Your order is being prepared".
- Bind a dynamic pop-up modal or custom Toast notification that triggers when the order status is updated by the kitchen to "Ready for pickup".

### 5. Chef Kitchen Dashboard (Dashboard.jsx)

FIFO Order Card Queue:

- Render a clean, scrollable column of active kitchen cards in a strict First-In, First-Out (FIFO) sequence.
- Each card must explicitly show: order details, table numbers, customized item choices, and timestamps.

State Progression Buttons (StatusButtons.jsx):

- Provide line chefs with clear click triggers to transition order states between 'preparing' and 'ready', dynamically broadcasting changes to the client.

### 6. Technical Engineering Requirements

- Use React functional components with modern Hooks (useState, useContext, useEffect).
- Wire the complete router infrastructure using React Router.
- Manage global states (Auth, Cart) cleanly via Context Providers, avoiding generic prop-drilling.
- Enforce strict Tailwind CSS variables for smooth layouts, avoiding unnecessary third-party styling packages.